



Nivell 1

Connecta Python amb MySQL Workbench i carrega les dades de la teva base de dades del Sprint 4 per utilitzar-les en tots els exercicis.

1. Connecta Python amb MySQL Workbench i carrega les dades de la teva base de dades del Sprint 4 per utilitzar-les en tots els exercicis.
2. Per a cada ítem, crea una visualització adequada segons les variables especificades. Interpreta els resultats segons les teves dades.

Recorda: **quan seleccionis les columnes, pensa sempre en el mètode que faràs servir i inclou les que calguin per a la funció de visualització que vulguis utilitzar.**

- Una variable numèrica.
- Dues variables numèriques.
- Una variable categòrica.
- Una variable categòrica i una numèrica.
- Dues variables categòriques.
- Tres variables combinades.
- Crea un Pairplot.

Nivell 1

Connecta Python amb MySQL Workbench i carrega les dades de la teva base de dades del Sprint 4 per utilitzar-les en tots els exercicis.

1. Connecta Python amb MySQL Workbench i carrega les dades de la teva base de dades del Sprint 4 per utilitzar-les en tots els exercicis.

```
#importar todo

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine

import mysql.connector
from matplotlib.ticker import FuncFormatter
import matplotlib.ticker as ticker
import matplotlib.cm as cm
import matplotlib.colors as mcolors
import plotly.express as px
```

✓ 0.0s

```
#conexion a mysql genera error con pandas

# conexion = mysql.connector.connect(
|   # user="root",password="1234",database="sprint4")

conexion = create_engine(
|   "mysql+mysqlconnector://root:1234@localhost/sprint4"
| )

print("Conexión exitosa")
```

✓ 0.0s

Conexión exitosa

#crear dataset

```
query_sinamount = """
SELECT
    transaction_product.transaction_id,
    transactions.timestamp,
    transactions.declined,
    transactions.user_id,
    data_users.name,
    data_users.surname,
    data_users.country AS user_country,
    data_users.city,
    transactions.business_id,
    companies.company_name,
    companies.country AS company_country,
    transaction_product.product_id,
    products.product_name,
    products.price AS product_price,
    products.colour,
    products.weight
FROM transaction_product
LEFT JOIN transactions
|   ON transaction_product.transaction_id = transactions.id

LEFT JOIN products
|   ON transaction_product.product_id = products.id

LEFT JOIN data_users
|   ON transactions.user_id = data_users.id

LEFT JOIN companies
|   ON transactions.business_id = companies.company_id;
"""

# Creo dataframe sin amount porque utilizo tabla productos y sumo los valores individuales de cada product_price en cada transaccion.
# SINO se repetia el valor por la cantidad de productos

df_sinamount = pd.read_sql(query_sinamount, conexion)
```

```
# Aseguro formato de algunas columnas para luego no tener problemas

# Fechas
df_sinamount["timestamp"] = pd.to_datetime(df_sinamount["timestamp"])

# Enteros
df_sinamount["user_id"] = df_sinamount["user_id"].astype("int64")
df_sinamount["product_id"] = df_sinamount["product_id"].astype("int64")

# Booleanos
df_sinamount["declined"] = df_sinamount["declined"].astype("bool")

# Numericos
df_sinamount["product_price"] = pd.to_numeric(df_sinamount["product_price"])
df_sinamount["weight"] = pd.to_numeric(df_sinamount["weight"])

# Strings
df_sinamount["user_country"] = df_sinamount["user_country"].astype("string")
df_sinamount["city"] = df_sinamount["city"].astype("string")
df_sinamount["company_name"] = df_sinamount["company_name"].astype("string")
df_sinamount["product_name"] = df_sinamount["product_name"].astype("string")

df_sinamount["year"] = df_sinamount["timestamp"].dt.year
df_sinamount["month"] = df_sinamount["timestamp"].dt.month
df_sinamount["day"] = df_sinamount["timestamp"].dt.day

# Exporto DataFrame a CSV
ruta = r"C:\Users\gabri\Desktop\Especializacion\python\SPRINT 11\df_sinamount.csv"

df_sinamount.to_csv(ruta, index=False)
```

```
# Creo la df_transacciones para calculos que necesite amount y no necesite datos de los productos y con declined False

df_transacciones = df_sinamount[df_sinamount["declined"] == False]

df_transacciones = df_transacciones.groupby("transaction_id").agg(
    timestamp=("timestamp", "first"),
    user_id=("user_id", "first"),
    user_country=("user_country", "first"),
    city=("city", "first"),
    company_country=("company_country", "first"),
    total_productos=("product_id", "count"),
    total_venta=("product_price", "sum")
).reset_index()

df_transacciones["timestamp"] = pd.to_datetime(df_transacciones["timestamp"])

df_transacciones["total_venta"] = pd.to_numeric(df_transacciones["total_venta"])

df_transacciones["total_productos"] = df_transacciones["total_productos"].astype("int64")

df_transacciones["year"] = df_transacciones["timestamp"].dt.year

df_transacciones["month"] = df_transacciones["timestamp"].dt.month

df_transacciones["day"] = df_transacciones["timestamp"].dt.day

#Exporto df
df_transacciones.to_csv(ruta, index=False)
```

Creo un KPI

```

import matplotlib.pyplot as plt
#kpi con total de transacciones
total_transacciones = df_transacciones["transaction_id"].count()

print("Total de transacciones:", total_transacciones)

plt.figure(figsize=(6,3))

# coloco un texto en cualquier posicion de la figura

plt.text(
    0.5,
    0.5,
    f"{total_transacciones:}",
    fontsize=40,
    ha="center",
    va="center",
    fontweight="bold"
)

plt.text(
    0.5,
    0.20,
    "Total de transacciones",
    fontsize=12,
    ha="center"
)

plt.axis("off")
plt.show()

```

✓ 0.1s

Total de transacciones: 99763



Grafico con Variable Numerica

```
# creo un df con ingresos y transacciones por year para usar con frecuencia

ventas_year = df_transacciones.groupby("year").agg(
    ingresos=("total_venta", "sum"),
    total_transacciones=("transaction_id", "nunique")
).reset_index()
ventas_year.to_csv(ruta, index=False)
```

[9] ✓ 0.1s



ventas_year

[10] ✓ 0.0s

...

	year	ingresos	total_transacciones
0	2015	2537626.57	9843
1	2016	2551692.87	9839
2	2017	2561608.52	9888
3	2018	2539997.34	9889
4	2019	2600809.41	9987
5	2020	2602211.63	10075
6	2021	2643099.45	10272
7	2022	2635489.23	10092
8	2023	2574960.75	9965
9	2024	2582888.65	9913

```

# Vuelvo a importar lo que vaya a utilizar aqui por cuestiones de aprendizaje

import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
import pandas as pd

# Ordenar (importante)
ventas_por_year = ventas_year.sort_values("year")

# Función formato moneda para valor eje
def moneda(x, pos):
    return f'{x:,.0f}'.replace(",", ".")

# Gráfico
plt.figure()

#dibujar grafico de lineas
plt.plot(
    ventas_por_year["year"],
    ventas_por_year["ingresos"],
    marker='o'
)

# Formato eje Y con FuncFormatter porque sino me ponía 1e6 o valores no muy legibles
plt.gca().yaxis.set_major_formatter(FuncFormatter(moneda))

#titulo y labels
plt.title("Ventas por Año")
plt.xlabel("Año")
plt.ylabel("Ingresos")

plt.grid() #cuadricula de fondo

plt.show()

```

11]

✓ 0.8s



Grafico con dos Variables Numericas


```
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
import pandas as pd

#crear figura
fig, ax1 = plt.subplots(figsize=(10, 5))

# Transacciones
ax1.plot(
    ventas_year["year"],
    ventas_year["total_transacciones"],
    marker="o",
    color="#1f77b4", # azul más pro
    linewidth=2,
    label="Transacciones"
)

ax1.set_xlabel("Año")
ax1.set_ylabel("Transacciones")
ax1.grid(alpha=0.3)
```

```
# Ingresos
ax2 = ax1.twinx()

ax2.plot(
    ventas_year["year"],
    ventas_year["ingresos"],
    marker="o",
    color="#2ca02c", # verde más pro
    linewidth=2,
    label="Ingresos"
)

ax2.set_ylabel("Ingresos")

# evitar notacion cinetica
ax2.ticklabel_format(style='plain', axis='y')

# Titulo
plt.title("Transacciones e Ingresos por Año", fontsize=13, weight="bold")
# Formato moneda para que se muestre bien
plt.gca().yaxis.set_major_formatter(FuncFormatter(moneda))

# eje izquierdo (transacciones)
ax1.set_ylabel("Transacciones", color="black", weight="bold")
ax1.tick_params(axis="y", labelcolor="#1f77b4")

# eje derecho (ingresos)
ax2.set_ylabel("Ingresos", color="black", weight="bold")
ax2.tick_params(axis="y", labelcolor="#2ca02c")

# Leyenda para el grafico
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()

ax1.legend(
    lines1 + lines2,
    labels1 + labels2,
    loc="upper left",
    frameon=False
)

plt.show()
✓ 0.4s
```

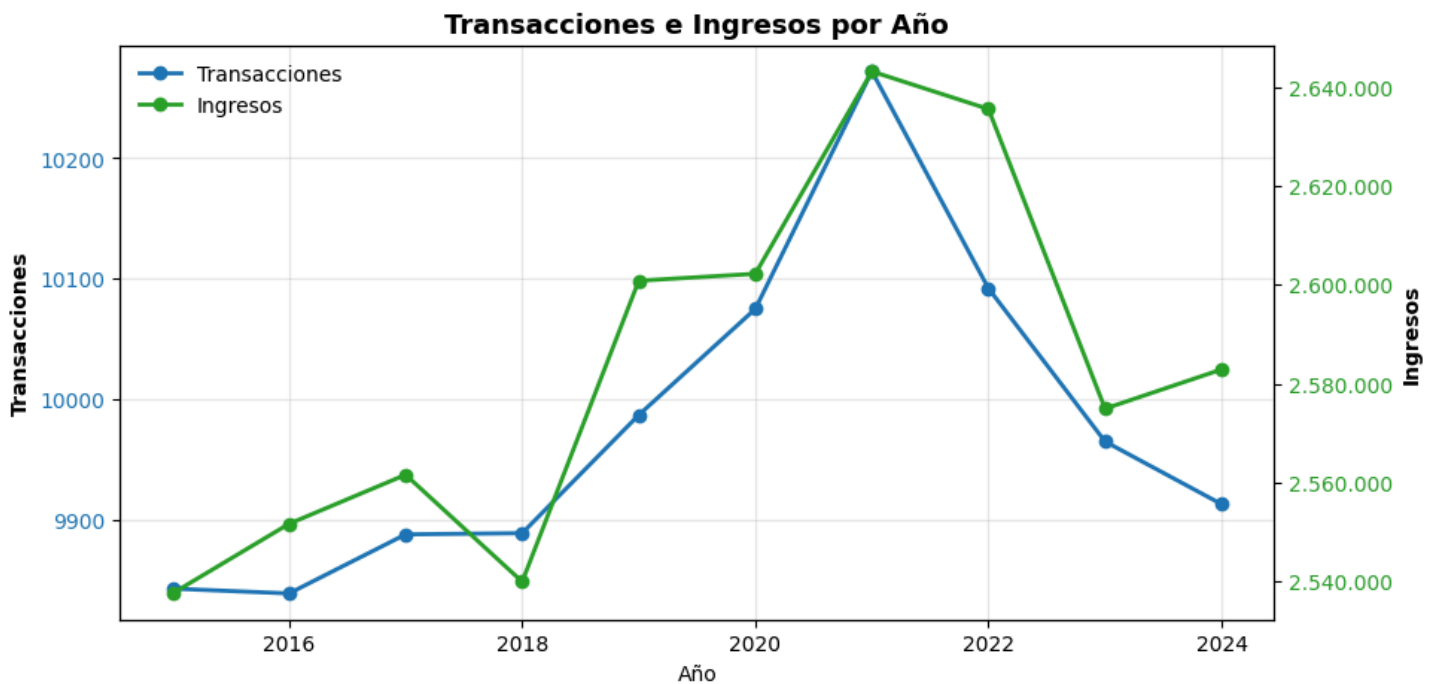


Grafico con Una variable Categorica

```

# creo df de pais y cantidad de empresas
empresas_country = (
    df_sinamount.groupby("company_country")["company_name"]
    .nunique()
    .sort_values(ascending=False)
    .reset_index(name="cantidad_empresas")
)

print(empresas_country)

```

15] ✓ 0.2s

	company_country	cantidad_empresas
0	Sweden	11
1	Netherlands	10
2	Italy	9
3	United Kingdom	9
4	United States	9
5	Belgium	8
6	Germany	8
7	Norway	7
8	Australia	6
9	Ireland	6
10	New Zealand	6
11	Canada	5
12	France	3
13	China	2
14	Spain	1

```

import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import matplotlib.cm as cm

valores = empresas_country["cantidad_empresas"]

# normalizar valores para el color map
norm = mcolors.Normalize(vmin=valores.min(), vmax=valores.max())

# colormap rojo amarillo verde
colors = cm.RdYlGn(norm(valores))

#crear figura
plt.figure(figsize=(10,6))
#crear barra horizontal
plt.barh(
    empresas_country["company_country"],
    valores,
    color=colors
)
#labels
plt.xlabel("Cantidad de empresas")
plt.ylabel("País")
plt.title("Cantidad de empresas por país")

#mostar alto arriba bajo abajo sino me lo hacia al reves
plt.gca().invert_yaxis()

# numeros al final de cada barra
for i, v in enumerate(valores):
    plt.text(v + 0.1, i, str(v), va="center")

plt.show()

```

✓ 0.2s

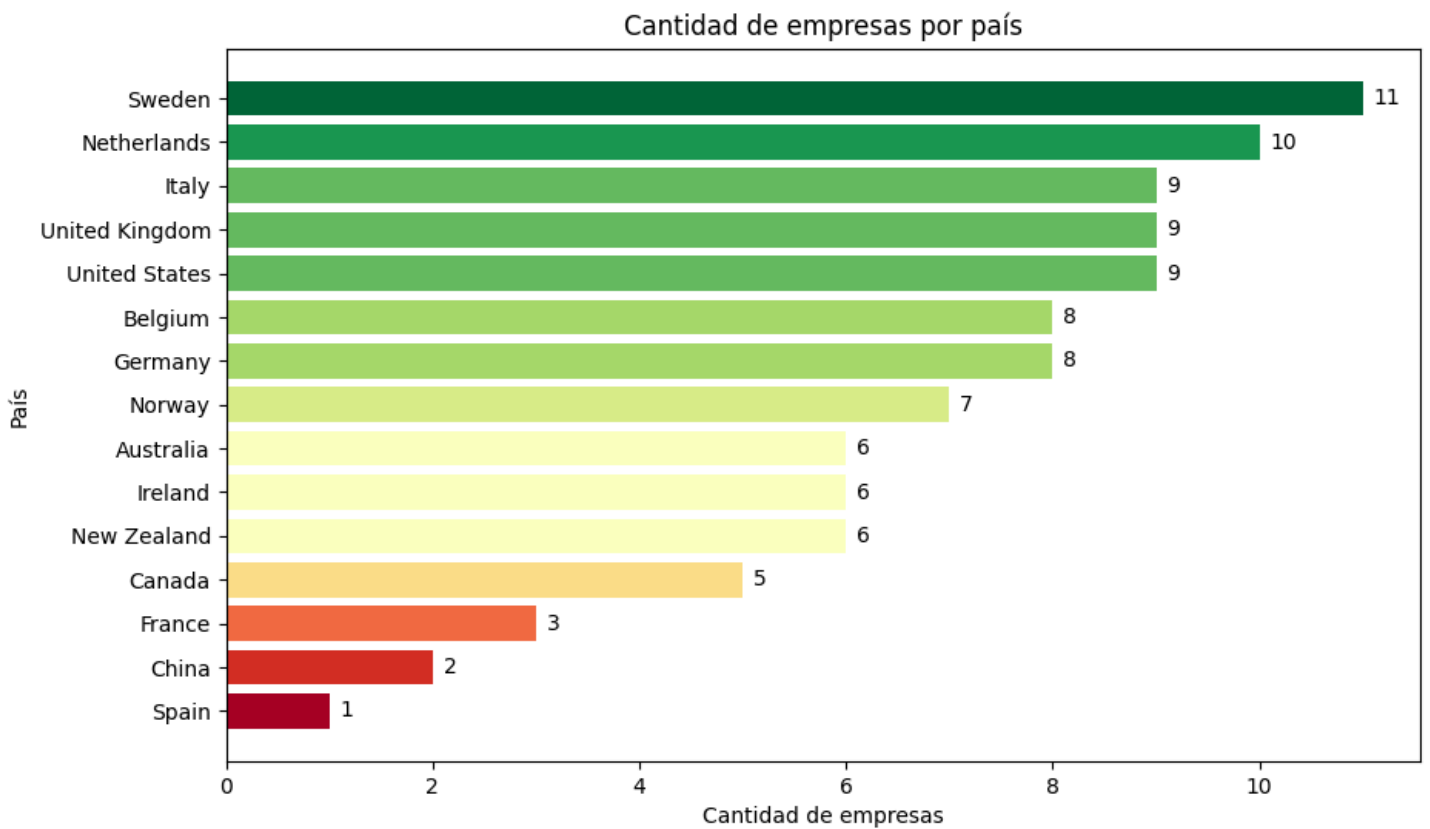


Grafico con Una variable Categorica y una Numerica

```
#creo figura barras con matplotlib

plt.figure(figsize=(10,6))
plt.bar(
    users_country["user_country"],
    users_country["cantidad_usuarios"]
)

plt.xlabel("País")
plt.ylabel("Cantidad de usuarios")
plt.title("Usuarios por país")

plt.xticks(rotation=45) #pongo los nombres del eje x en 45 grados para que entre mejor en el grafico

plt.show()
```

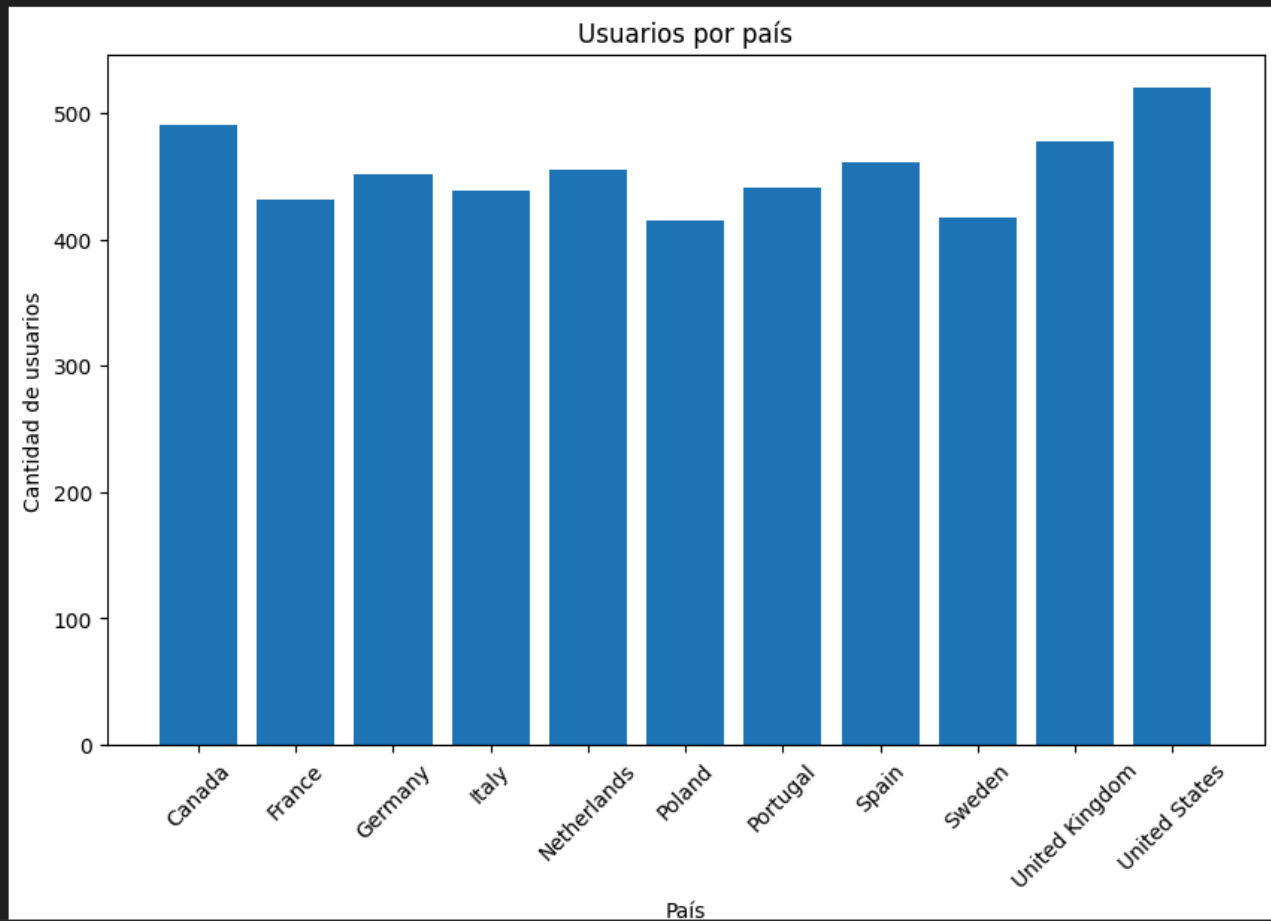


Grafico con Dos variables Categoricas

```
# Contar cantidad
producto_pais = df_sinamount.groupby(
    ["user_country", "product_name"]
).size().reset_index(name="cantidad")

# Top producto por país
producto_pais_top = (
    producto_pais
    .sort_values("cantidad", ascending=False)
    .groupby("user_country")
    .head(1)
)

# Ordenar para gráfico
producto_pais_top = producto_pais_top.sort_values("cantidad", ascending=True)
#nombre productos
productos = producto_pais_top["product_name"].unique()

# Colores
colors = plt.cm.Set3(np.linspace(0, 1, len(productos)))

color_dict = dict(zip(productos, colors)) #diccionario producto colores
bar_colors = producto_pais_top["product_name"].map(color_dict)
```

```
# Gráfico
plt.figure(figsize=(10,6))

bars = plt.barh(
    producto_pais_top["user_country"],
    producto_pais_top["cantidad"],
    color=bar_colors
)

# Etiquetas (producto)
for bar, producto in zip(bars, producto_pais_top["product_name"]):
    plt.text(
        bar.get_width(),
        bar.get_y() + bar.get_height()/2,
        f" {producto}",
        va="center"
    )

plt.xlabel("Cantidad vendida")
plt.ylabel("País")
plt.title("Producto más vendido por país (cantidad)")

plt.tight_layout()
plt.show()

✓ 0.6s
```

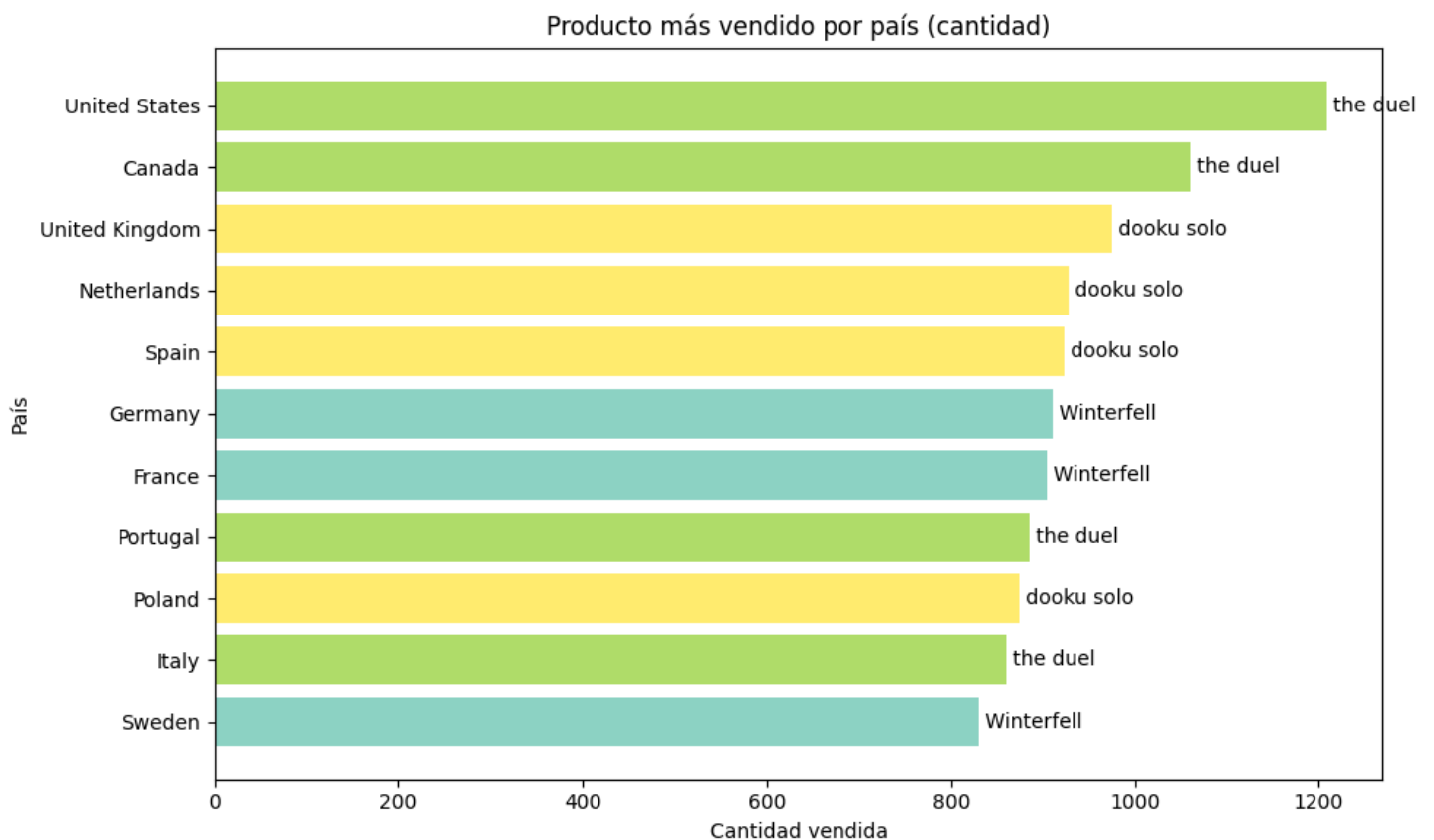


Grafico Tres variables Combinadas

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
# tabla de valores --- Utilizo df_transacciones para contar las transacciones
tabla_count = pd.crosstab(
    df_transacciones["user_country"],
    df_transacciones["company_country"]
)
#tabla de porcentajes
tabla_pct = pd.crosstab(
    df_transacciones["user_country"],
    df_transacciones["company_country"],
    normalize="index"
) * 100

#creo labels para las anotaciones del heatmap, en cantidad y porcentjaes
label_heatmap = tabla_pct.round(1).astype(str) + "%\n(" + tabla_count.astype(str) + ")"

#creo fdigura
plt.figure(figsize=(12,6))

sns.heatmap(
    tabla_pct,
    annot=label_heatmap,
    fmt="",
    cmap="viridis"
)

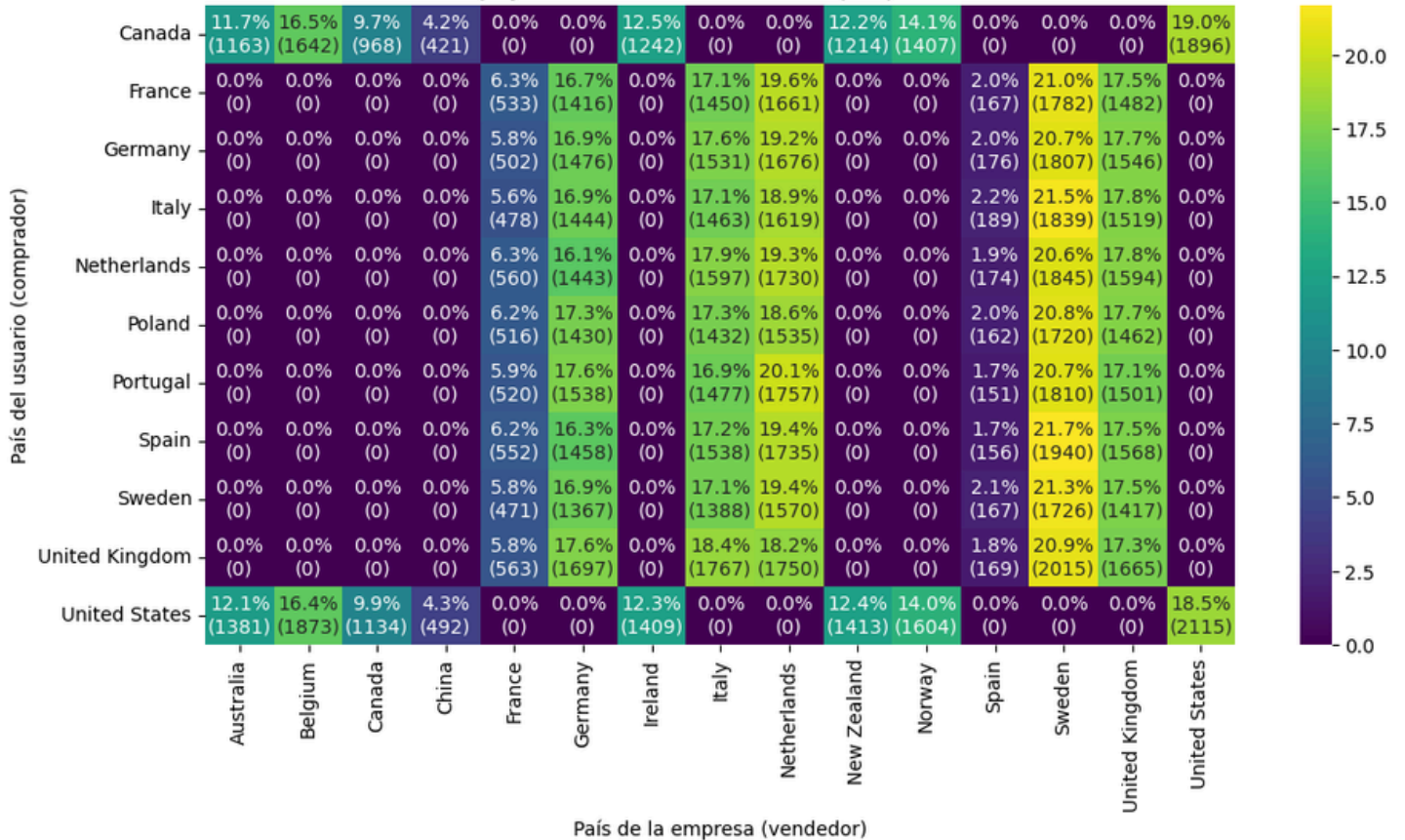
plt.xlabel("País de la empresa (vendedor)")
plt.ylabel("País del usuario (comprador)")
plt.title("Porcentaje y cantidad de transacciones por pais de usuario")

plt.show()

```

✓ 2.1s

Porcentaje y cantidad de transacciones por país de usuario



PairPlot

```

#creo un df para el pairplot
df_cliente = df_sinamount.groupby("user_id").agg(
    ventas_totales=("product_price", "sum"),
    total_transacciones=("transaction_id", "nunique"),
    productos_comprados=("product_id", "count"),
    diversidad_productos=("product_id", "nunique")
).reset_index()

#creo columnas con metricas
df_cliente["ticket_medio"] = df_cliente["ventas_totales"] / df_cliente["total_transacciones"]

df_cliente["productos_por_transaccion"] = df_cliente["productos_comprados"] / df_cliente["total_transacciones"]

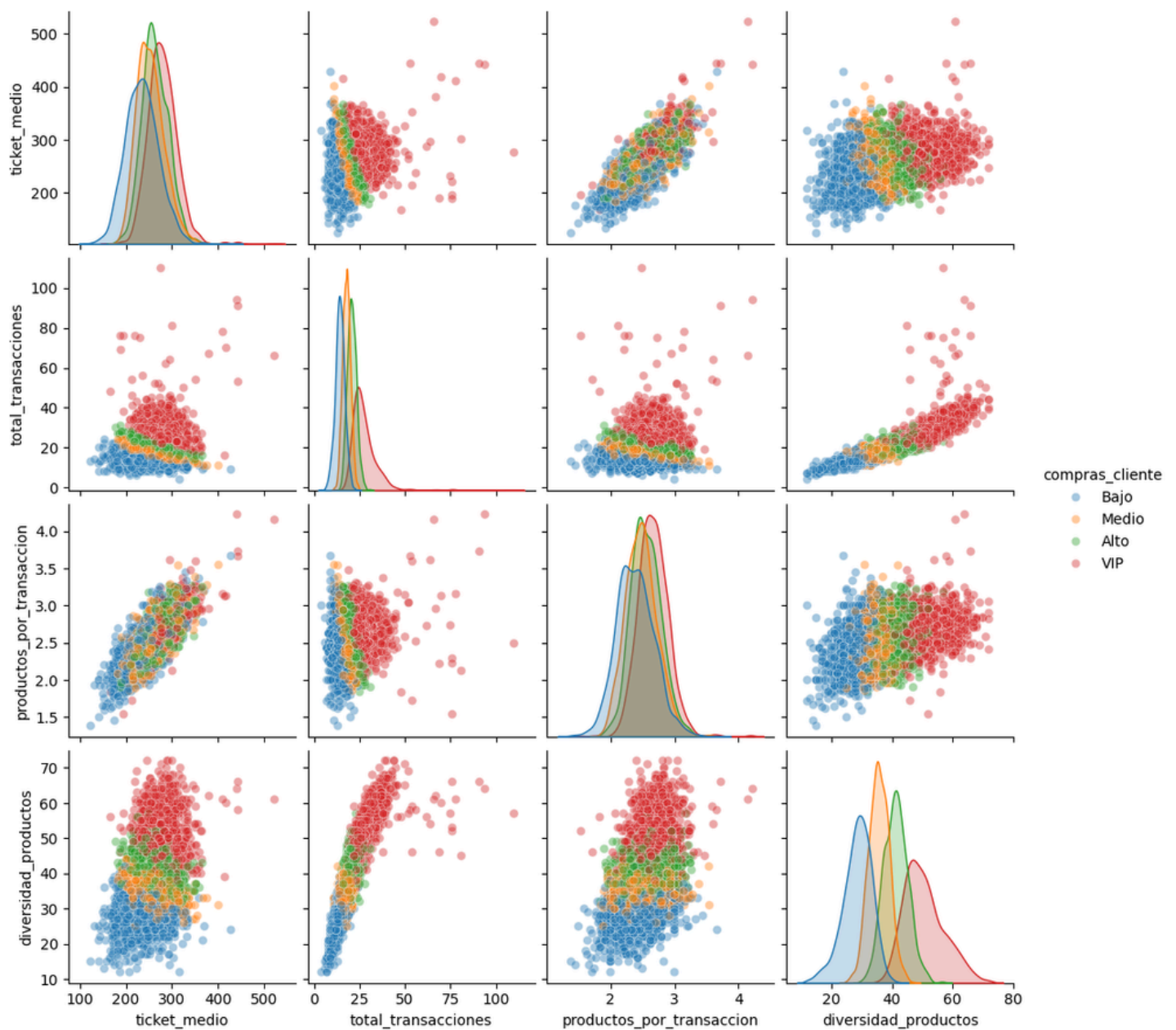
df_cliente = df_cliente[df_cliente["total_transacciones"] > 1]

#creo segmentacion de cliente por ventas totales
df_cliente["compras_cliente"] = pd.qcut(
    df_cliente["ventas_totales"],
    q=4,
    labels=["Bajo", "Medio", "Alto", "VIP"]
)

#creo el pairplot con las columnas de las metricas
sns.pairplot(
    df_cliente,
    vars=[
        "ticket_medio",
        "total_transacciones",
        "productos_por_transaccion",
        "diversidad_productos"
    ],
    hue="compras_cliente",
    plot_kws={"alpha": 0.4}
)

```

✓ 13.0s



Nivell 2

1. Representa la correlació d'algunes variables i interpreta els resultats segons les teves dades.
2. Implementa un Jointplot per explorar la relació entre dues variables i interpreta els resultats segons les teves dades.

```

import seaborn as sns
import matplotlib.pyplot as plt

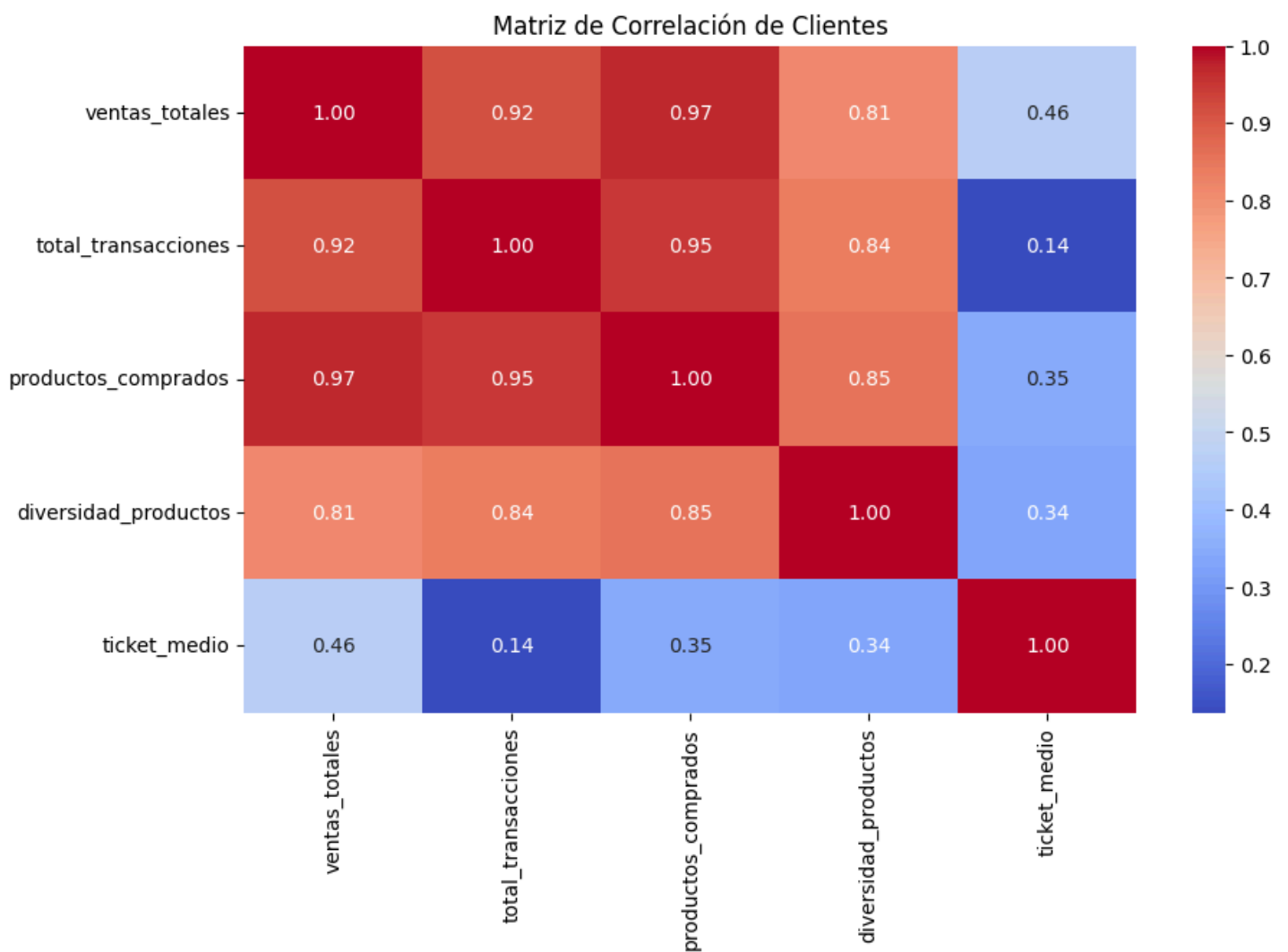
# Selección de variables
columnas = [
    "ventas_totales",
    "total_transacciones",
    "productos_comprados",
    "diversidad_productos",
    "ticket_medio"
]

# Matriz de correlación
correlacion = df_cliente[columnas].corr()

# Heatmap
plt.figure(figsize=(10,6))
sns.heatmap(correlacion, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Matriz de Correlación de Clientes")
plt.show()

```

✓ 0.4s



Ventas y volumen de compra

Las ventas totales estan muy relacionadas con la cantidad de productos vendidos y el numero de transacciones. Esto significa que cuanto mas compran los clientes (mas veces o mas productos), mas dinero generan.

Comportamiento de los clientes activos

Los clientes que compran mas veces tambien compran mas productos y mas variedad. Es decir, no solo compran seguido, sino que tambien prueban diferentes productos.

Diversidad de productos

La diversidad de productos tambien esta bastante relacionada con las compras. Esto indica que los clientes no se quedan con un solo tipo de producto, sino que suelen variar bastante.

Ticket medio

El ticket medio no tiene una relacion fuerte con las demas variables. Esto quiere decir que el dinero que se gasta en cada compra no es lo mas importante en este caso. El crecimiento de las ventas no viene de tickets altos sino de la frecuencia y cantidad de transacciones.

Conclusion:

En general, el negocio depende mas de cuantas veces compran los clientes y cuantos productos compran, que de cuanto gastan en cada compra. Es decir, funciona mas por volumen que por compras grandes.

Ademas, hay una oportunidad de mejorar el ticket medio, ya que ahora mismo no esta aportando tanto como podria.

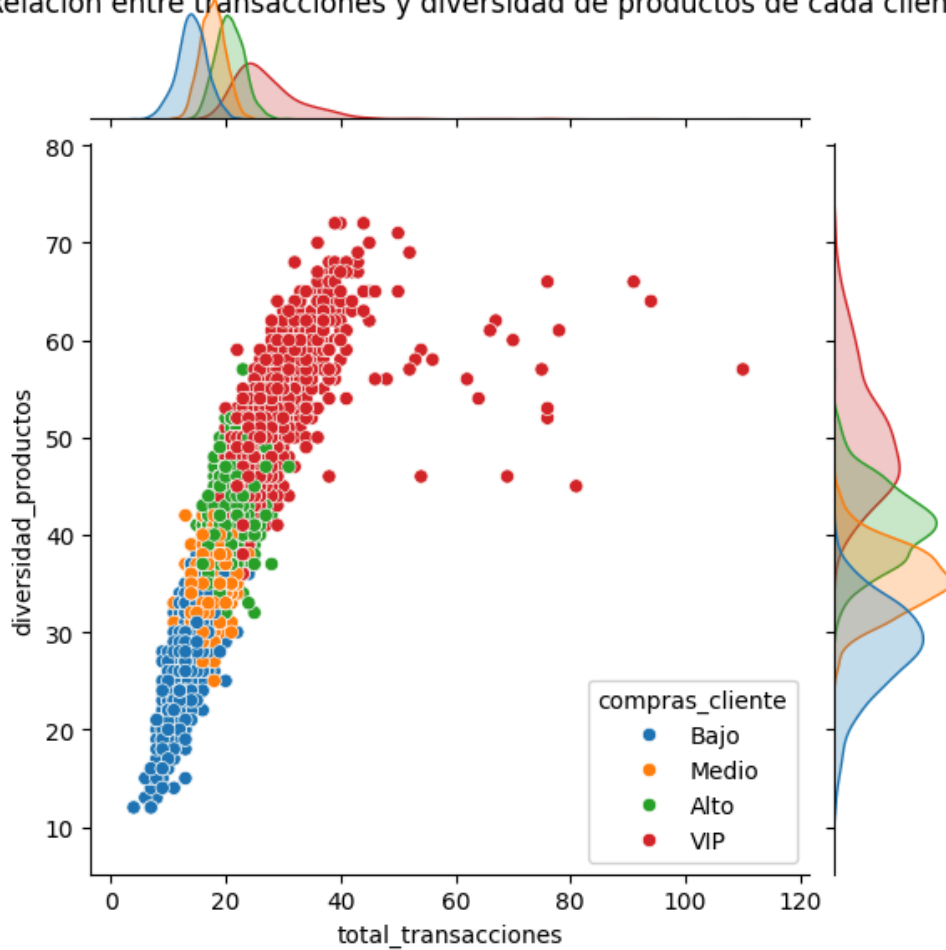
JointPlot

```
#crear joinplot
```

```
jointplot_clientes = sns.jointplot(  
    data=df_cliente,  
    x="total_transacciones",  
    y="diversidad_productos",  
    hue="compras_cliente", # use los colores del segmento valor  
    kind="scatter" # reg no funciona con hue  
)  
jointplot_clientes.fig.suptitle("Relacion entre transacciones y diversidad de productos de cada cliente")  
  
plt.show()
```

✓ 13s

Relacion entre transacciones y diversidad de productos de cada cliente





Nivell 3

1. Transfereix totes les visualitzacions del Nivell 1 a Power BI utilitzant scripts de Python.

Recorda: **quan carreguis els teus dataframes a Power BI, assegura't d'incloure una columna identificadora o una combinació de columnes que garanteixi la unicitat de cada registre. Per defecte, Power BI elimina duplicats i podries perdre informació.**

Obtener datos

scr

×

Todo

Otras

Todo

Script de R

Script de Python

Conectores certificados

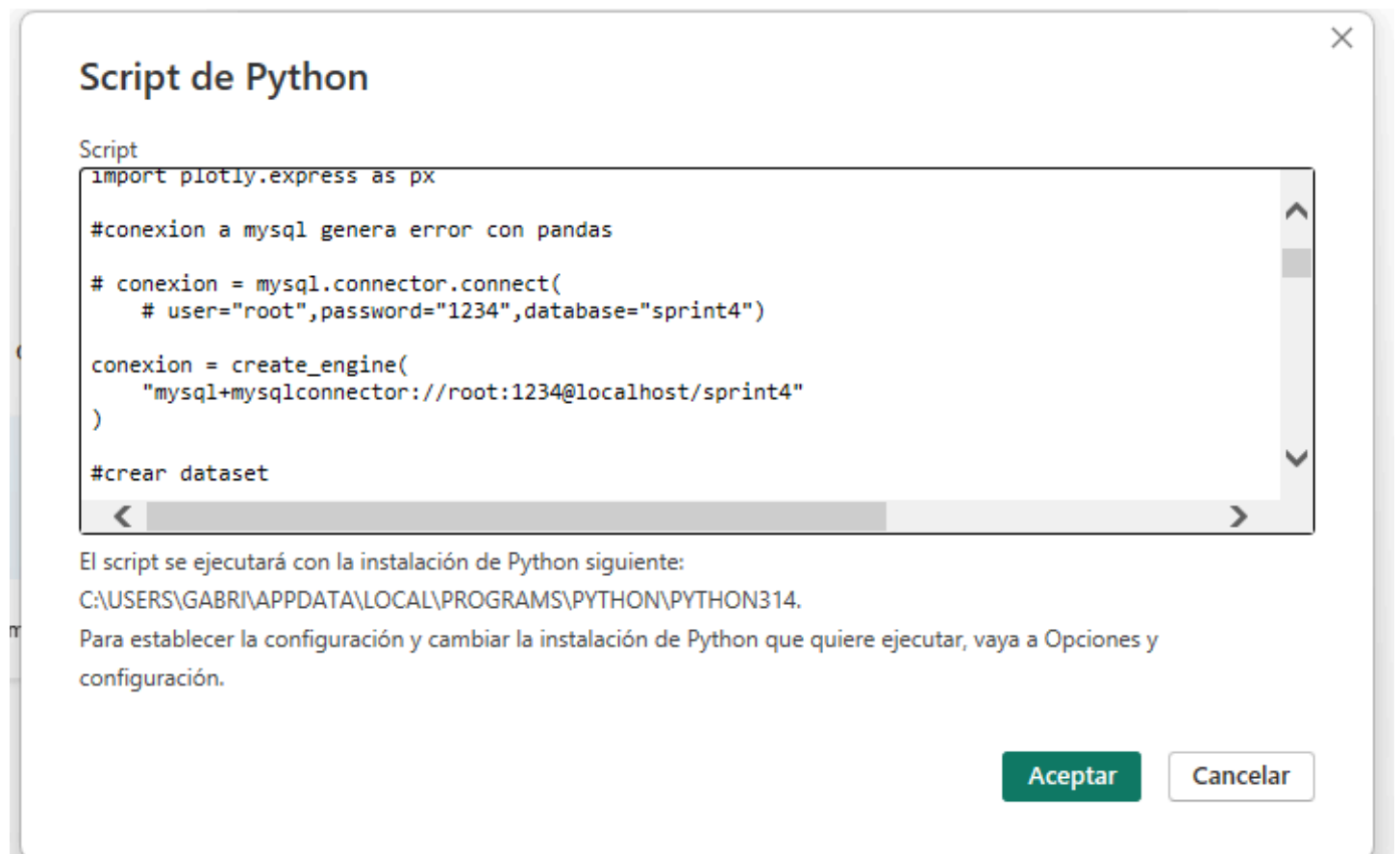
Aplicaciones de plantilla

Conectar

Cancelar

A continuacion copio la primer parte del codigo de python donde importo las librerias, me conecto con mysql y creo los dataframe. Con ese codigo, power bi me dara como resultado los

dataframes como tablas



Tambien cargo los dataframes que creo mas adelante, para asi hacer mas facil los graficos. aunque igualmente podria hacerlos en cada scrip de cada grafico, no se que es mejor.

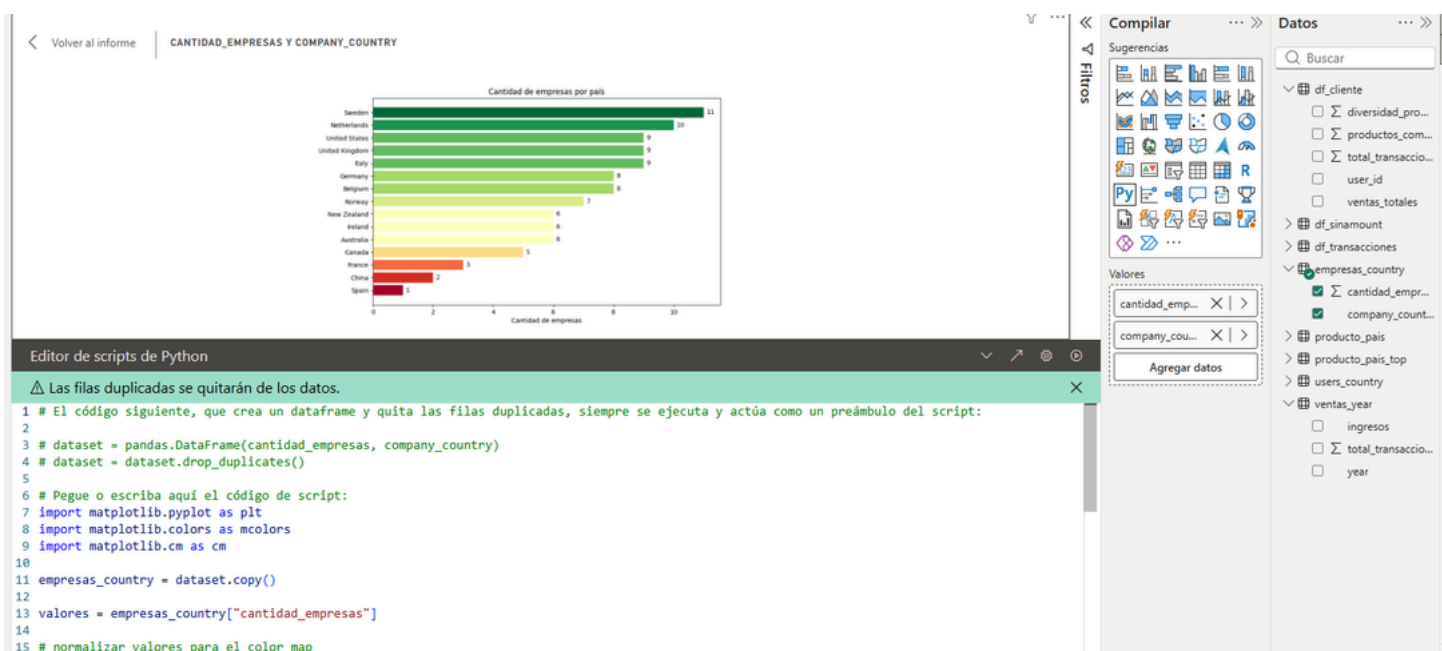
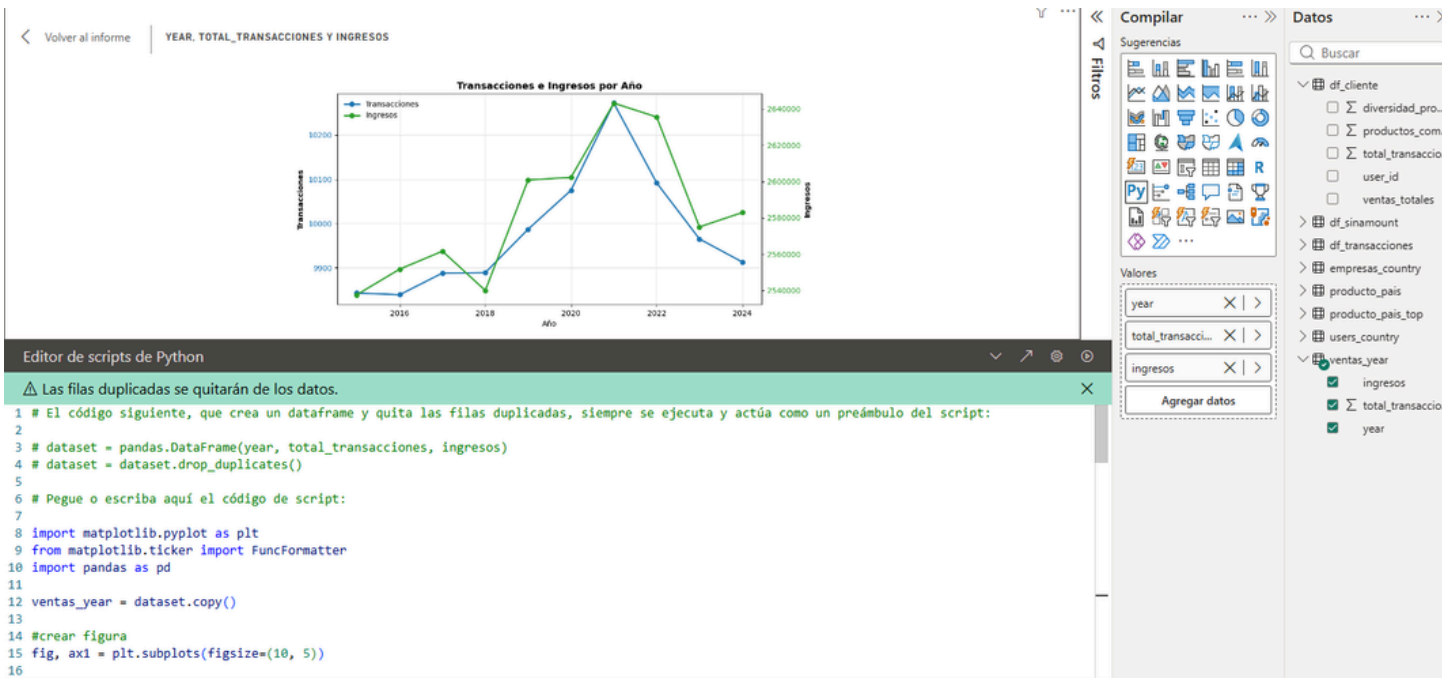
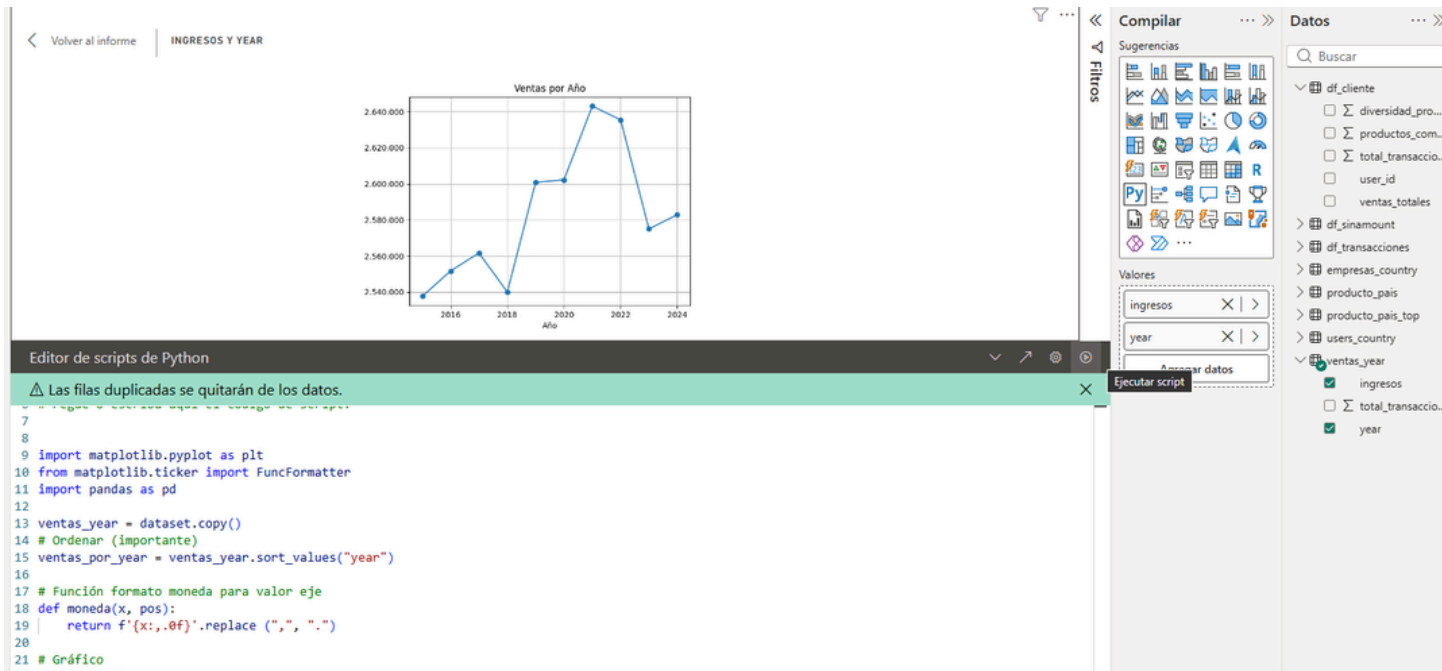
Se podria exportar todos los dataframe en python y cargarlos como CSV tambien



Pongo transformar datos ya que como se puede ver en product_price los valores no estan correctos, habra que corregir el formato regional.

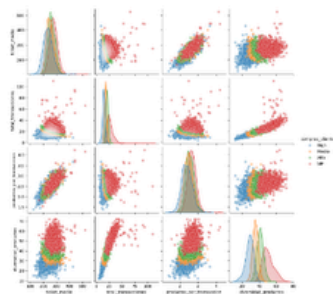
[illegible]

Ya con las tablas cargadas, pongo un objeto visual de python y me aparece el editor de scripts de Python. En el pegare el mismo codigo que en python para hacer el grafico, lo unico que pondre que el dataframe `ventas_year` que utilizaba en python para este grafico ahora es `dataset.copy()`, que utiliza los valores que agregue en el compilado de power bi, ingresos e year.



Volver al informe

VENTAS_TOTALES, USER_ID, TOTAL_TRANSACCIONES, PRODUCTOS_COMPRADOS Y DIVERSI...



Editor de scripts de Python

⚠ Las filas duplicadas se quitarán de los datos.

1 # El código siguiente, que crea un dataframe y quita las filas duplicadas, siempre se ejecuta y actúa como un preámbulo del script:

2

3 # dataset = pandas.DataFrame(ventas_totales, user_id, total_transacciones, productos_comprados, diversidad_productos)

4 # dataset = dataset.drop_duplicates()

5

6 # Pegue o escriba aquí el código de script:

7 import matplotlib.pyplot as plt

8

9 import pandas as pd

10 import seaborn as sns

11

12 df_cliente = dataset.copy()

13

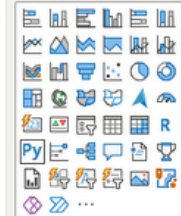
14 #creo columnas con metricas

15 df_cliente["ticket_medio"] = df_cliente["ventas_totales"] / df_cliente["total_transacciones"]

16

Compilar

Sugerencias



Valores

ventas_totales	X		>
user_id	X		>
total_transacci...	X		>
productos_co...	X		>
diversidad_pro...	X		>
Agregar datos			

Datos

Buscador

